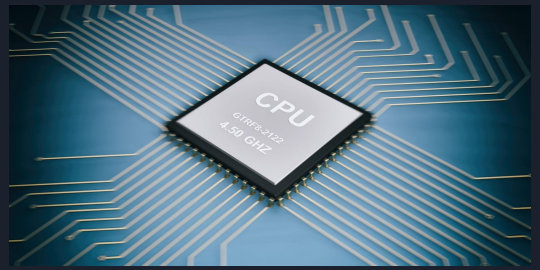
Workload-Driven Performance Analysis of Transformers on CPU and GPU Architectures

Daniel Sanguino

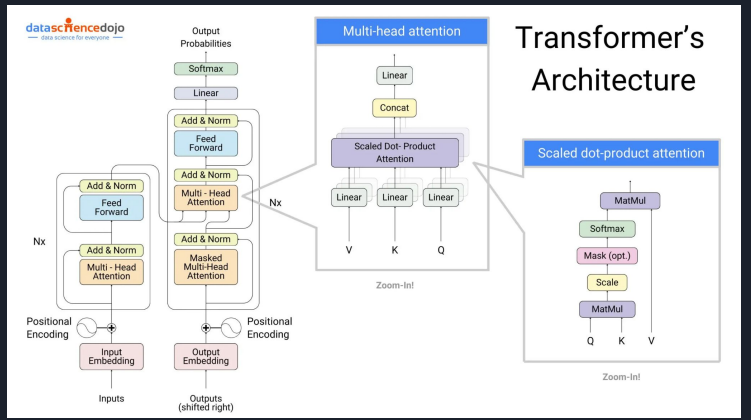
Project Motivation

- Transformer workloads have become central to modern AI systems
- Performance on hardware varies significantly depending on the workload size such as batch size
- Understanding where and why these crossovers occur is essential for architectural optimization
- CPUs often excel at low-parallelism workloads
- GPUs dominate when workloads become large enough to keep parallel compute units busy



Background Information

- Transformers rely heavily on matrix multiplications in self-attention and feedforward layers
- These operations parallelize extremely well -> GPU benefit
- CPU performs competitively for small workloads due to low overhead
- Batch size – number of samples processed per inference step, and acts as the primary workload



Problem Statement

Research Question: How do transformer workloads scale on CPU vs. GPU, and at what workload size does GPU performance surpass CPU performance?

Why it matters:

- Helps determine when CPU inference is more efficient than GPU inference
- Informs decisions in resource-constrained environments such as small labs or embedded systems
- Highlights architectural trade-offs between throughput, latency, and utilization

Methodology

ASLFR model - transformer encoder trained on ASL frame sequences

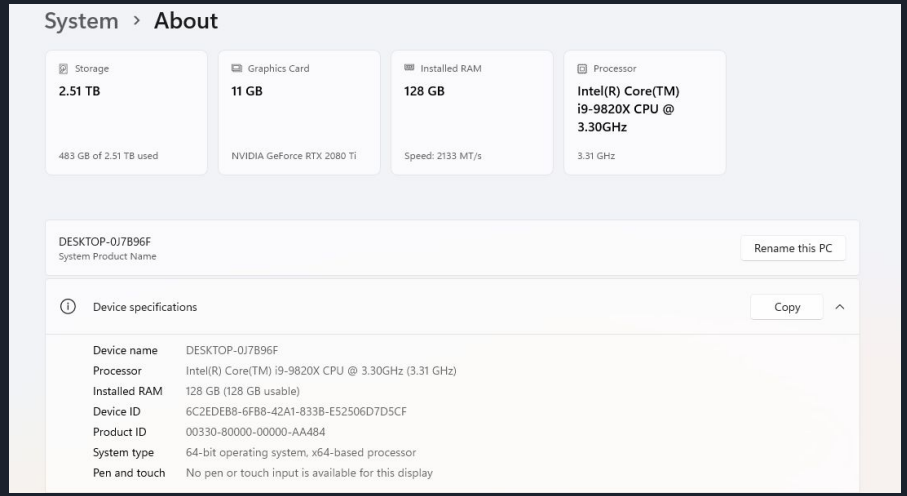
Task: Benchmarking inference

Workload Variable: Batch size sweep

Hardware: Intel I9-9820X + 2080 TI

Metrics collected:

- Throughput (samples/sec)
- Latency (ms/batch)
- CPU utilization
- GPU utilization
- Memory usage (DRAM/VRAM)



Benchmarking Procedure

1. Preprocess ASLFR features from exported Kaggle dataset
2. Load TensorFlow transformer as SavedModel
3. Run inference on CPU and GPU across batch sizes
4. Measure latency, throughput, and utilization
5. Plot metrics

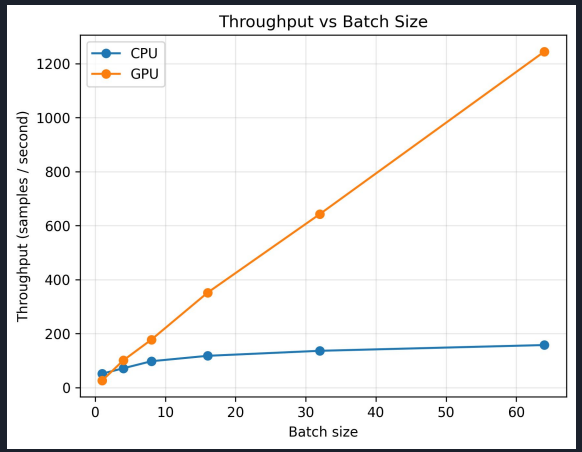
```python
def main():
    args = parse_args()
    out_dir = _ensure_out_dir(args.out_dir)

    cpu_df, gpu_df = load_results(
        cpu_csv=args.cpu_csv,
        gpu_csv=args.gpu_csv,
        combined_csv=args.combined_csv,
    )

    # If both are empty, nothing to do
    if cpu_df.empty and gpu_df.empty:
        print("[WARN] No data to plot. Check your CSV paths.")
        return

    # Generate plots
    plot_throughput(cpu_df, gpu_df, out_dir)
    plot_latency(cpu_df, gpu_df, out_dir)
    plot_cpu_utilization(cpu_df, gpu_df, out_dir)
    plot_gpu_utilization(gpu_df, out_dir)

    print(f"[INFO] All plots saved under: {out_dir}")


if __name__ == "__main__":
    main()
```

```python
def benchmark_inference(
    device: str,
    model_path: str,
    X: np.ndarray,
    batch_sizes: List[int],
    num_iters: int,
    num_warmup: int,
) -> List[Dict[str, Any]]:
    """
    Run inference on the given device for multiple batch sizes.
    Supports X with shape:
      - (N, D)
      - (N, T, D)
    """

    import tensorflow as tf

    print("[INFO] TensorFlow version:", tf.__version__)
    print("[INFO] Available physical devices:", tf.config.list_physical_devices())

    print(f"[INFO] Loading model from {model_path}")
    infer_fn = load_aslfr_inference_callable(model_path)

    n_samples = X.shape[0]
    X_ndim = X.ndim
    print(f"[INFO] Input tensor ndim = {X_ndim}, shape = {X.shape}")

    results: List[Dict[str, Any]] = []
    rng = np.random.default_rng(seed=42)

    for batch_size in batch_sizes:
        if batch_size > n_samples:
            print(
                f"[WARN] Batch size {batch_size} > number of samples {n_samples}. "
                f"Skipping this batch size."
            )
            continue

        print(f"\n[INFO] ==== Benchmarking batch_size = {batch_size} on {device.upper()} ====")

        indices = rng.choice(
            n_samples,
            size=batch_size * (num_warmup + num_iters),
            replace=True,
        )

        if X_ndim == 2:
            X_batched = X[indices].reshape(num_warmup + num_iters, batch_size, -1)
        elif X_ndim == 3:
            T = X.shape[1]
            D = X.shape[2]
            X_batched = X[indices].reshape(num_warmup + num_iters, batch_size, T, D)
        else:
            raise ValueError(f"Unsupported input ndim={X_ndim}; expected 2 or 3.")

        print(f"[INFO] Warmup: {num_warmup} batches (not timed)")
        for i in range(num_warmup):
            infer_fn(X_batched[i])

        cpu_metrics_before = get_cpu_metrics()
        if device == "gpu":
            gpu_metrics_before = get_gpu_metrics()
        else:
            gpu_metrics_before = {
                "gpu_util_percent": float("nan"),
                "gpu_mem_used_gb": float("nan"),
            }

        print(f"[INFO] Timed runs: {num_iters} batches")
        start = time.perf_counter()
        for i in range(num_warmup, num_warmup + num_iters):
            infer_fn(X_batched[i])
        end = time.perf_counter()

        total_time = end - start
        total_batches = num_iters
        total_samples = num_iters * batch_size
        avg_batch_latency_ms = (total_time / total_batches) * 1000.0
```

Throughput vs. Batch Size

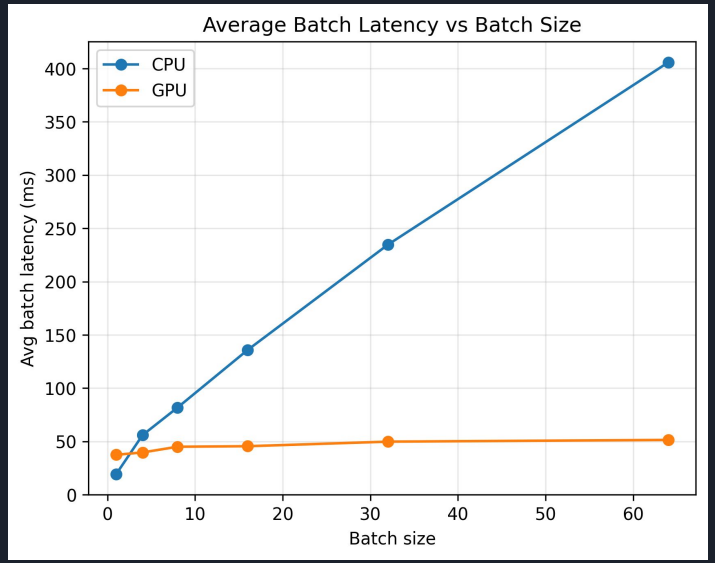
- GPU throughput increases almost linearly with batch size
- CPU throughput grows slowly and plateaus early

GPU throughput scales nearly linearly because this model's compute cost increases proportionally with the batch size, and GPU parallel cores remain underutilized but active



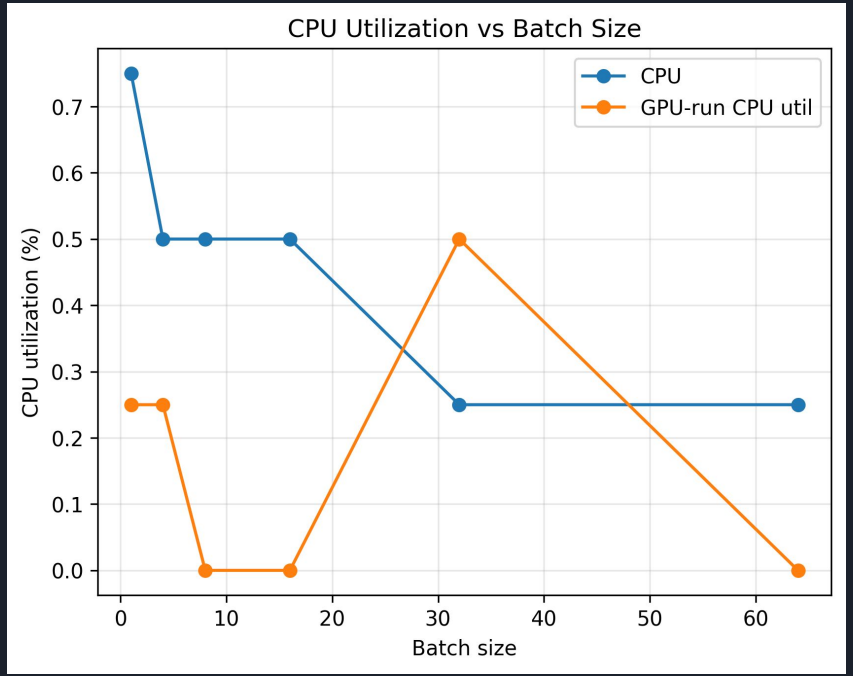
Latency vs. Batch Size

- CPU latency grows sharply as batch size increases
- GPU latency stays nearly constant around ~40-50ms
- GPU provides more stable and predictable inference
- Large batch sizes significantly degrade CPU responsiveness



CPU Utilization Scaling

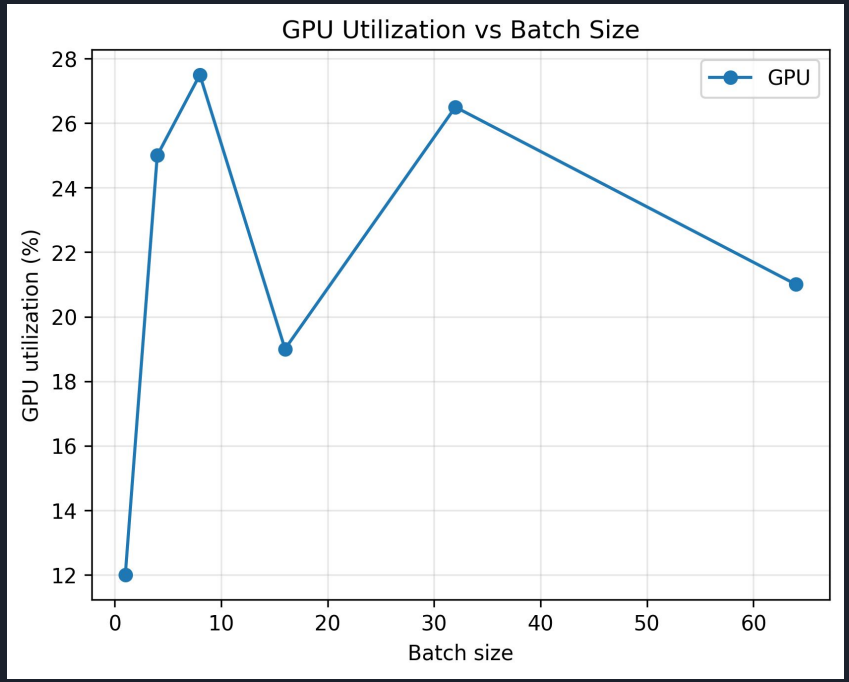
- CPU inference consumes a large portion of CPU resources
- During GPU inference, CPU load is minimal (mostly scheduling overhead)
- GPU offloads nearly all compute, freeing CPU resources
- CPU spike at batchsize=32 due to increased host-side work (tensor prep. + GPU feed) and measurement sampling catching that activity



GPU Utilization Scaling

- GPU utilization ranges only 12% to 27% (model is small)
- Even with low utilization, GPU throughput far exceeds CPU
- Model is not large enough to fully saturate GPU compute units

Despite low utilization, GPU outperforms CPU due to massive parallelism and matrix multiplication operations even at small workloads



Bottleneck Analysis

Small Workloads:

- CPU wins due to near-zero launch overhead
- GPU underutilized, dominated by kernel launch and data transfer overhead

Large Workloads:

- GPU parallel cores become saturated
- CPU bottlenecked by memory bandwidth and core limits

The ASLFR model is small enough that GPU is never fully saturated hence modest utilization

Architectural Interpretation

CPU Strengths:

- High single-thread performance
- Low latency for small workloads
- Good for small batches

GPU Strengths:

- Massive parallelism
- High throughput for large workloads
- Best suited for large batch sizes and high compute density

Obstacles & Challenges

1. Framework Limitations
    a. ASLFR model implemented in TensorFlow, preventing use of PyTorch Profiler, which supports more detailed kernel and operation-level analysis
    b. Had to rely on external tools (e.g. nvidia-smi, psutil) to gather utilization metrics
2. GPU Benchmarking on Windows
    a. TensorFlow GPU support on Windows is limited and difficult to configure
    b. Install WSL (windows subsystem for linux) 2 + Ubuntu , configure CUDA inside Linux, and run all benchmarks through WSL
3. Model formatting
    a. Original .h5 model lacked proper inference signatures which required conversion to SavedModel format

Future Work

1. Vary Sequence Length and Model Size
    a. Different workload dimensions such as sequence length or hidden dimension size may yield new CPU-GPU crossover behaviors
2. Expand to Training Phases
    a. Including forward + backward passes and optimization steps would illustrate full hardware utilization across the complete training cycle
3. Evaluate Additional Transformer Models
    a. Testing different models would improve generality and enable multi-model comparisons
4. Explore Hybrid CPU-GPU Scheduling
    a. Some recent research explores dynamically shifting workloads between CPU and GPU depending on batch size, memory pressure, or energy efficiency

Conclusion

- Workload size dramatically impacts CPU vs. GPU performance
- CPU is superior for small workloads due to minimal overhead
- GPU becomes dominant once workload size saturates compute units
- Even with low GPU utilization, GPU throughput dominates, showing that transformer inference benefits significantly from parallel hardware
- Small batch sizes are used when latency is more important than throughput (real-time systems)

Thank you!